IA8 — Deployment Readiness

Client Registration Feature — Inventory Management System

A. Cover Information

Full Name	Joy Ampadu
Student ID	22379034
Team Name	*JJ HUB*
Project Title	Inventory Management System
Feature Covered	Client Registration
Individual GitHub Classroom Repo URL	_____
Team Project Repository URL	https://github.com/Dept-of-Comp-Sci-University-of-Ghana/dcit208-client-project-2026-team-engineering-repository-seg26-48-j-j-hub
Related Issue / PR / Release Links	https://github.com/Dept-of-Comp-Sci-University-of-Ghana/dcit208-client-project-2026-team-engineering-repository-seg26-48-j-j-hub

B. Feature and Deployment Context

Feature description:

The Client Registration feature allows staff of the Inventory Management System to register a new client (customer or supplier) into the system. It captures the client's name, contact details, business/organization name, and account status, validates the input, and stores the record in the PostgreSQL database. Once registered, the client becomes available for linking to stock transactions, orders, and reports elsewhere in the system.

User story:

As an inventory staff member, I want to register a new client with their contact and business details so that I can associate future stock transactions with a verified client record.

Deployment status:

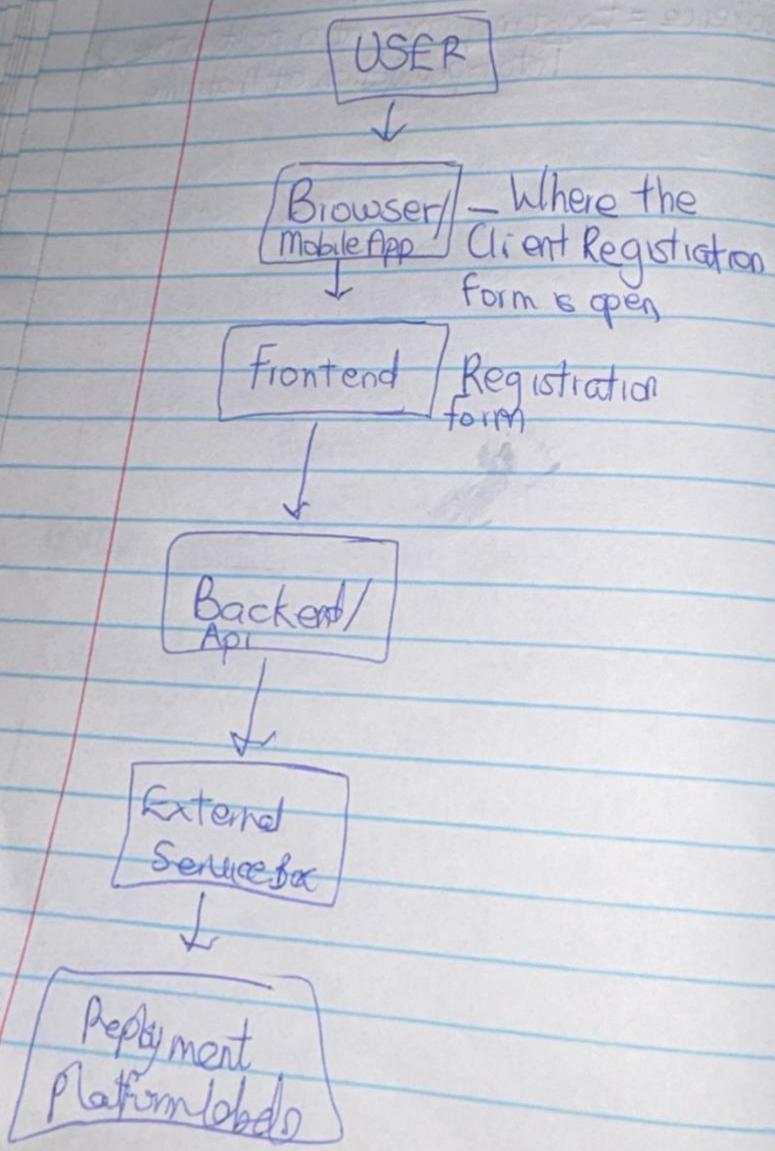
This feature is currently working locally and is staging-ready. The backend API and PostgreSQL schema are complete and tested locally; the frontend registration form is integrated and functional against the local API. Staging deployment (Vercel for the frontend, Render for the backend/API, and a managed Render PostgreSQL instance) has been configured but full staging verification is [COMPLETE]. The deployment steps below reflect exactly what is needed to take the current local build to staging.

C. Environment and Configuration Table

Variable	Purpose	Secret? / Placeholder Value
DATABASE_URL	PostgreSQL connection string for the API	*Yes*
PORT	Port the Express server listens on	No — 5000
NODE_ENV	Runtime environment flag	No — production
JWT_SECRET	Signing key for staff login/auth tokens	*Yes* — ••••••••••••
CORS_ORIGIN	Allowed frontend origin for API requests	*No — https://[your-app].vercel.app*
VITE_API_BASE_URL	Frontend's base URL for the deployed API	*No — https://[your-api].onrender.com*

D. Personal Hand-Drawn Deployment Sketch

Joy Ampadu
22379031
17/08/2026
Client Registration



E. Deployment Steps

Backend / API

1. Clone the repository: git clone [YOUR REPO URL]
2. Install dependencies: cd server && npm install
3. Set environment variables (DATABASE_URL, PORT, NODE_ENV, JWT_SECRET, CORS_ORIGIN) in Render's dashboard for the API service.
4. Run database migrations: npm run migrate (or npx knex migrate:latest / npx prisma migrate deploy — depending on your ORM).
5. Start the server locally to verify: npm start — confirm it boots on the configured PORT with no errors.
6. Push to the main/staging branch connected to Render; Render auto-builds and deploys the API.
7. Confirm the health endpoint responds, e.g. GET https://[your-api].onrender.com/api/health

Frontend

8. cd client && npm install
9. Set VITE_API_BASE_URL in Vercel's project environment variables to point at the deployed API.
10. Build locally to verify: npm run build
11. Push to the branch connected to Vercel; Vercel auto-builds and deploys.
12. Visit the deployed URL and confirm the Client Registration form loads.

Deployment URL(s): Frontend — [PASTE VERCEL URL] | API — [PASTE RENDER URL]

F. Smoke-Test Checklist

- Valid input: Submit the registration form with all required fields correctly filled — expect HTTP 201 and the new client visible in the client list.
- Invalid input: Submit with a missing required field (e.g. blank client name) — expect HTTP 400 and a clear validation error message, no record created.
- Invalid input: Submit with an invalid phone/email format — expect HTTP 400 with a field-specific error.
- Duplicate check: Attempt to register a client with an existing unique identifier (e.g. email) — expect HTTP 409 or a duplicate-prevention message.
- Access control: Attempt to reach the registration endpoint without a valid staff login/token — expect HTTP 401/403, no record created.
- System error expectation: Temporarily point the API at an unreachable database (or stop the DB) and submit the form — expect a graceful HTTP 500 with a generic error message, not a raw stack trace exposed to the user.
- Persistence check: After a valid registration, refresh the client list page and confirm the new client still appears (confirms it was actually persisted, not just shown optimistically).

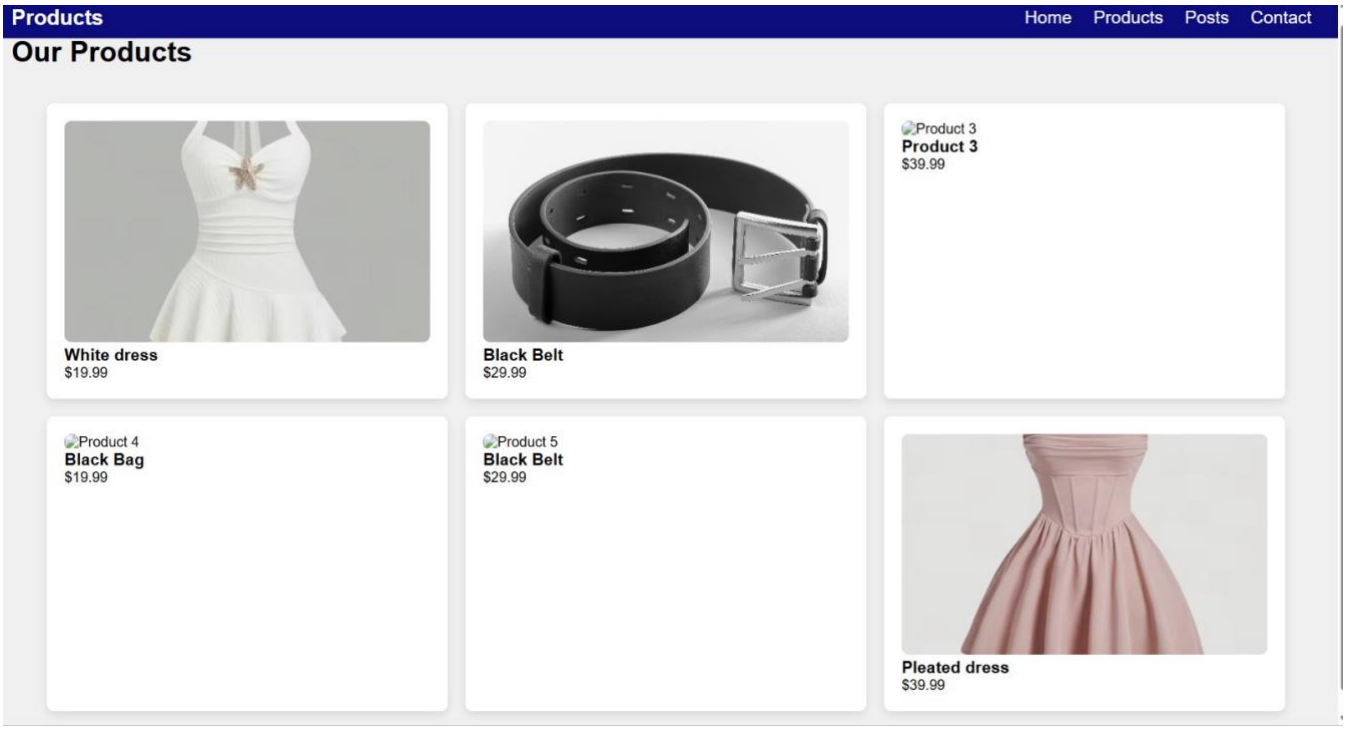
G. Failure and Rollback Walkthrough

Scenario: After deploying a new version of the API, client registration starts failing for all users.

Symptom	Users see "Registration failed" on every submission; the form returns HTTP 500 instead of 201.
Likely cause	The latest deploy included a database migration that renamed a column (e.g. phone → phone_number), but the API code deployed alongside it still references the old column name in one query path, or the migration failed to run on the staging database.

Evidence to check	Render service logs for the stack trace/SQL error; confirm the migration status (npx knex migrate:status or equivalent) on the staging DB; check the Render deploy log for a failed migration step.
Immediate response	Acknowledge the incident, put the registration form behind a brief maintenance notice on the frontend if possible, and stop further writes that could create inconsistent data.
Rollback / corrective action	Roll back the API service on Render to the previous successful deploy (Render "Redeploy previous version"). If the migration already partially ran, run the corresponding down-migration to restore the previous column name, or apply a corrective migration once the code fix is ready.
Confirm recovery	Re-run the smoke-test checklist in section F end-to-end, particularly the valid-input and persistence checks, and confirm no HTTP 500s appear in the logs over a short monitoring window.

H. Release Evidence



I. Operations and Handoff Note

Logs: The API logs each registration attempt (success/failure) with timestamp and status code via Render's built-in log stream; these are the first place to check after a deploy.

Health checks: A GET /api/health endpoint confirms the API and database connection are both reachable; this should be checked immediately after every deploy.

Backups: The staging PostgreSQL database relies on Render's automated daily backups; there is no separate manual backup step for this feature specifically.

Support notes: Duplicate client detection is based on email/phone uniqueness — support staff fielding "can't register client" complaints should first check whether the client already exists.

J. Oral Defence Video

https://youtu.be/lLEmYo7XFLg?si=VNOSTtFBkRDWpZrl

K. AI Disclosure

AI (Claude) was used to help structure and draft this deployment readiness document based on the assignment brief and the details I provided about my project (feature, stack, and deployment status). I personally verified the deployment steps, environment variables, and smoke-test checklist against my actual project setup before submission, and I wrote the failure/rollback scenario based on a realistic risk in my own deployment process. I did not allow AI to invent evidence, generate the hand-drawn sketch, record the oral defence video, or fabricate GitHub links, commit hashes, or release evidence — all of these were supplied and confirmed by me directly.
